

Deep Learning Optimisé - Jean Zay

Optimizing large models – part 1



INSTITUT DU
DÉVELOPPEMENT ET DES
RESSOURCES EN
INFORMATIQUE
SCIENTIFIQUE

cc BY NC SA

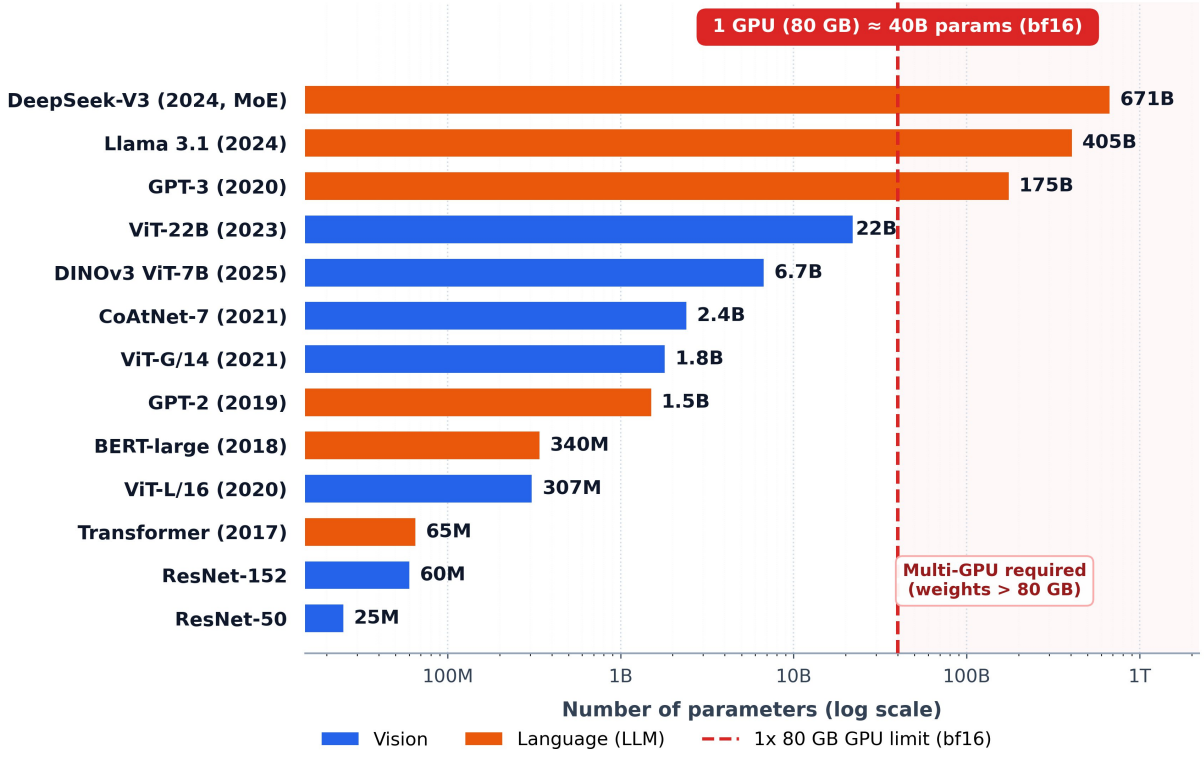
The Very Large Models

Transformers ◂

Vision Transformers (ViT) ◂

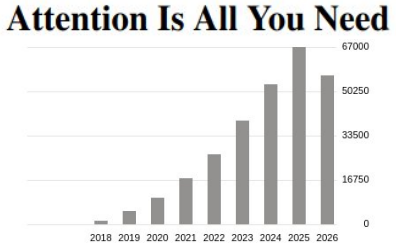
CoAtNet ◂

Larger and larger models



- Vision: from 60 M (ResNet-152) to several billions
- Language: hundreds of billions of parameters
- Reference: 1 GPU 80 Go ≈ 40 B parameters in BF16 ... weights only!

The first Transformer (Vaswani et al.,2017)

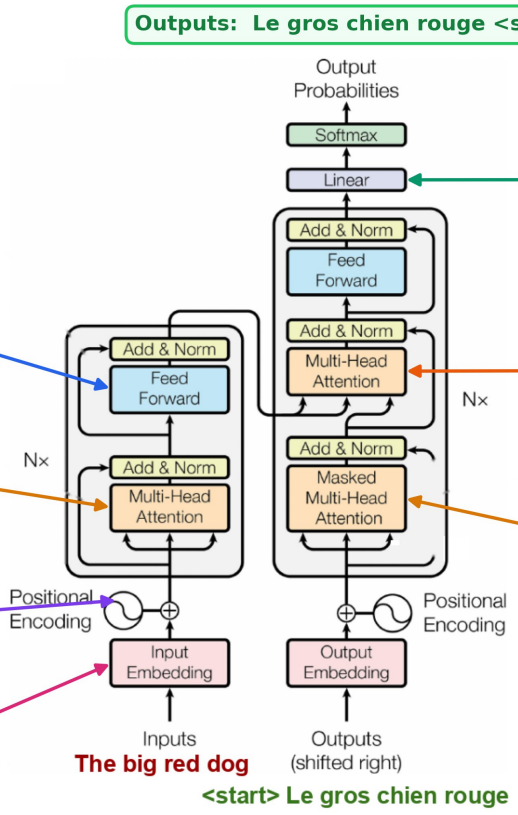


2-Layer MLP (Feed-Forward)
Applied to each word independently

Encoder Self-Attention
Each word attends to all source words

+ Position Encoding
Adds sequence order (otherwise: bag of words)

Words → Vectors (Embeddings)
Input tokens mapped to dense vectors



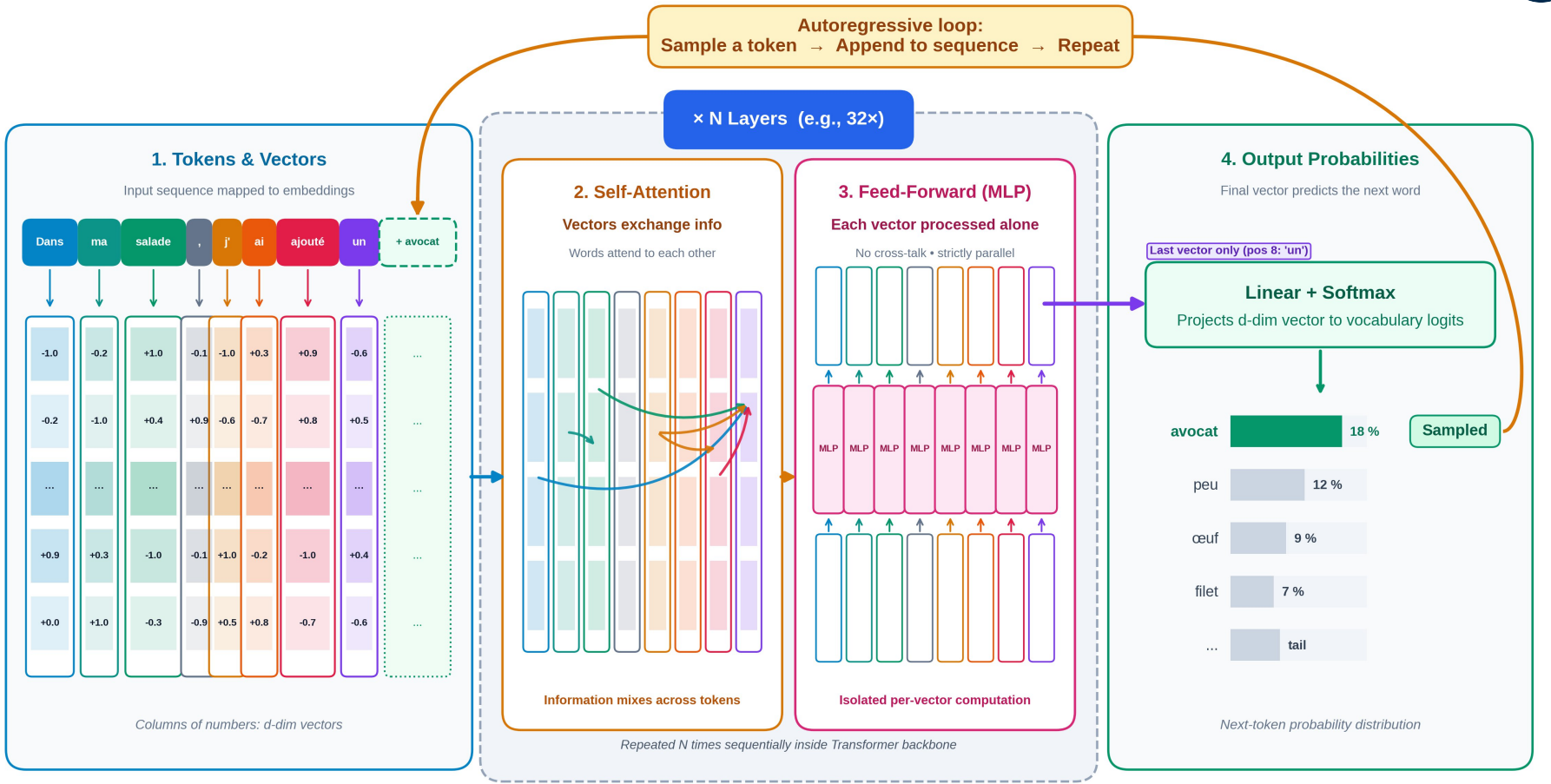
Linear + Softmax
Next-word probabilities over full vocabulary

Cross-Attention
Decoder queries the source sentence

Masked Attention (Decoder)
Attends only to past translated words

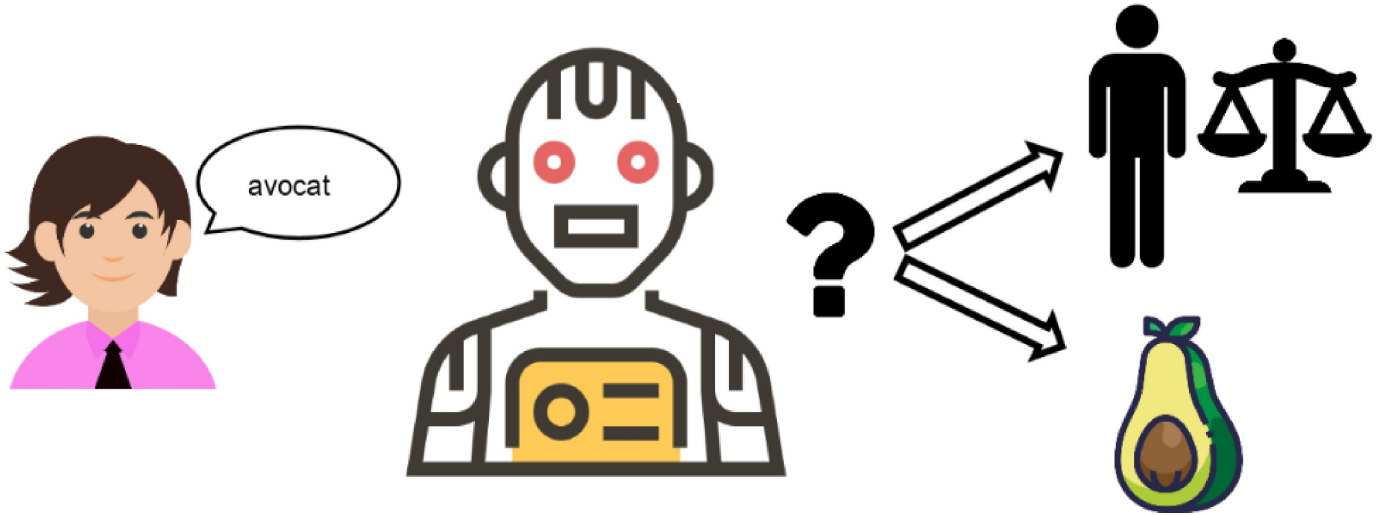
No more recurrence → Entire sequence processed in parallel → Ideal for GPUs

Transformer in one slide



Summary: Tokens become vectors → Attention mixes context → MLP transforms features → Linear + Softmax predicts next token

Attention: intuition (1/2)



Ambiguity: which meaning without context?

The Problem with Static Embeddings

1. Fixed token representation

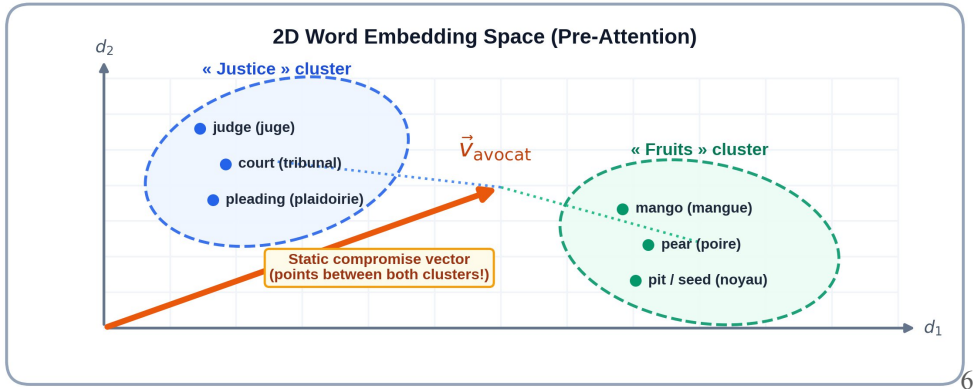
Before attention, each word has exactly one static vector.

2. Lexical polysemy

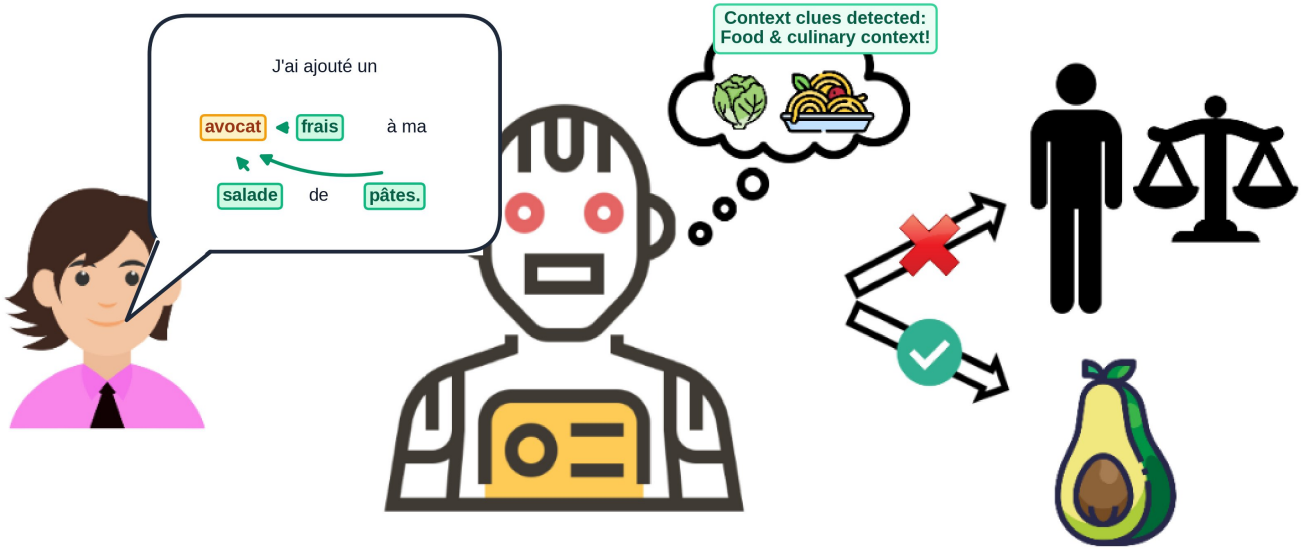
In French, ‘avocat’ means both lawyer and avocado.

3. Absence of sentence context

The model doesn’t know if we are in court or in the kitchen!



Attention: intuition (2/2)



Lawyer (justice)

[Discarded]
Incompatible with food context

Avocado (fruit)

[Resolved meaning]
Matches ‘frais’, ‘salade’, ‘pâtes’

How Self-Attention Solves Polysemy

1. Contextual attention score

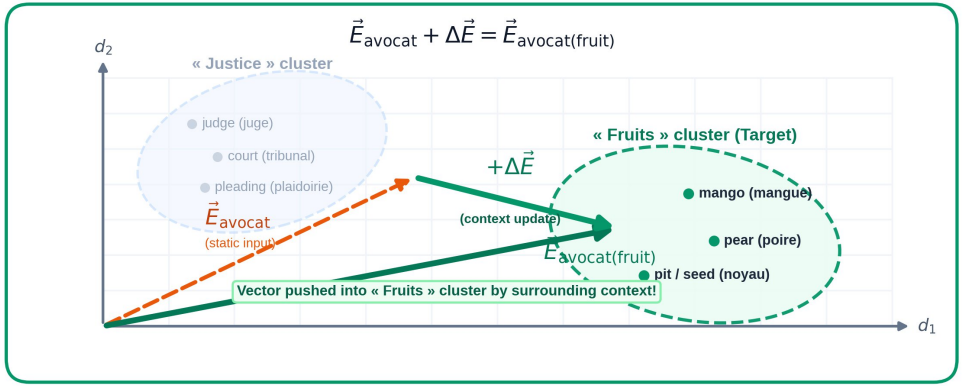
The word ‘avocat’ attends heavily to ‘frais’, ‘salade’, ‘pâtes’.

2. Context update vector (ΔE)

A correction vector ΔE is computed by blending context values.

3. Disambiguated representation

Adding ΔE shifts ‘avocat’ directly into the culinary fruit cluster!

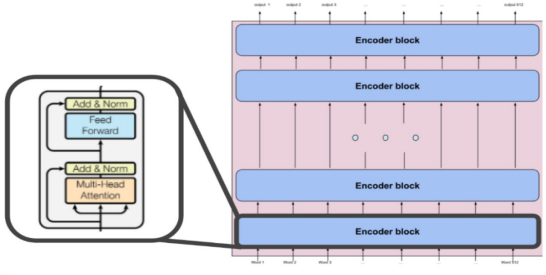


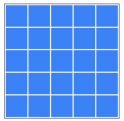
Three families of Transformers

Encoder-only

Auto-encoding • Bidirectional

BERT (2018), RoBERTa, DeBERTa, ViT





Full Matrix

Bidirectional Attention

Every token attends to all tokens

Visibility / Context
Sees full sentence:
bidirectional context across all tokens

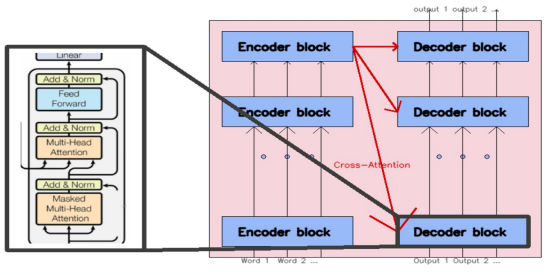
Pre-training Objective
Masked Language Modeling (MLM):
predict hidden tokens ([MASK])

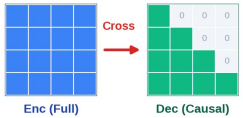
Primary Applications
Classification, Search, Embeddings:
NLU tasks, sentence scoring... and ViT

Encoder–Decoder

Sequence-to-Sequence (Seq2Seq)

Transformer (2017), T5, BART, Whisper





Full (Enc) + Causal (Dec)

Cross-Attention Link

Enc reads prompt • Dec writes

Visibility / Context
Encoder reads, Decoder writes:
cross-attention connects input to output

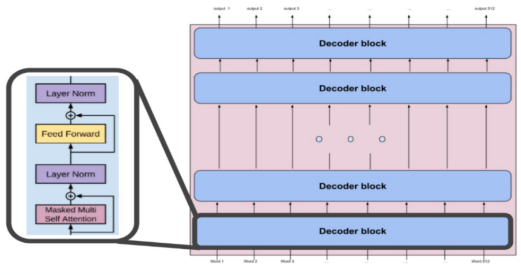
Pre-training Objective
Seq2seq / Denoising:
reconstruct corrupted spans & translation

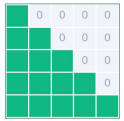
Primary Applications
Translation, Summarization:
Speech-to-text (Whisper), multimodal

Decoder-only

Auto-regressive • Causal

GPT-4, Llama 3, Mistral, Qwen, DeepSeek…





Causal Mask (Triangular)

Autoregressive Masking

Tokens only attend to past ($j \leq i$)

Visibility / Context
Only sees past tokens:
causal masking (strictly autoregressive)

Pre-training Objective
Next-token prediction (CLM):
predict token t+1 given past context

Primary Applications
Text generation, Chat, Coding:
reasoning & generalist AI via prompting

Self-Attention: computation

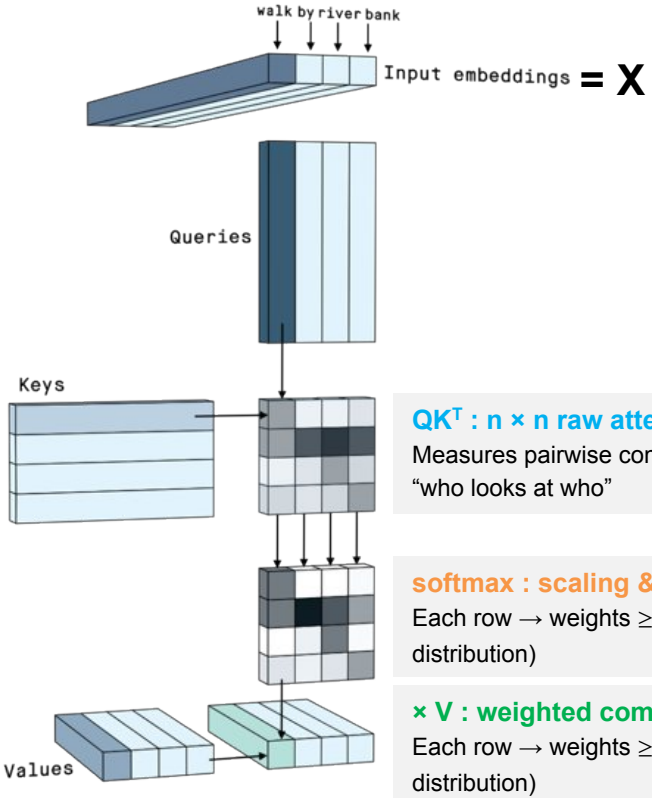
$$Q = \mathbf{X}W_Q,\ K = \mathbf{X}W_K,\ V = \mathbf{X}W_V$$
$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T/\sqrt{d_k})\ V$$

QUERY (Q)
"What am I looking for?"
e.g. for "bank" (or "avocado"): "a clue about my meaning?"

KEY (K)
"What do I contain?"
e.g. for "river" (or "salad"): "nature / waterway!"

VALUE (V)
"What do I transmit if selected?"
the contextual representation passed to the enrich the attended token

+ **DECODER** Causal attention masking
– Future attention score set to $-\infty$
– Tokens only attend to current and past positions



QK^T : n × n raw attention scores
Measures pairwise compatibility, "who looks at who"

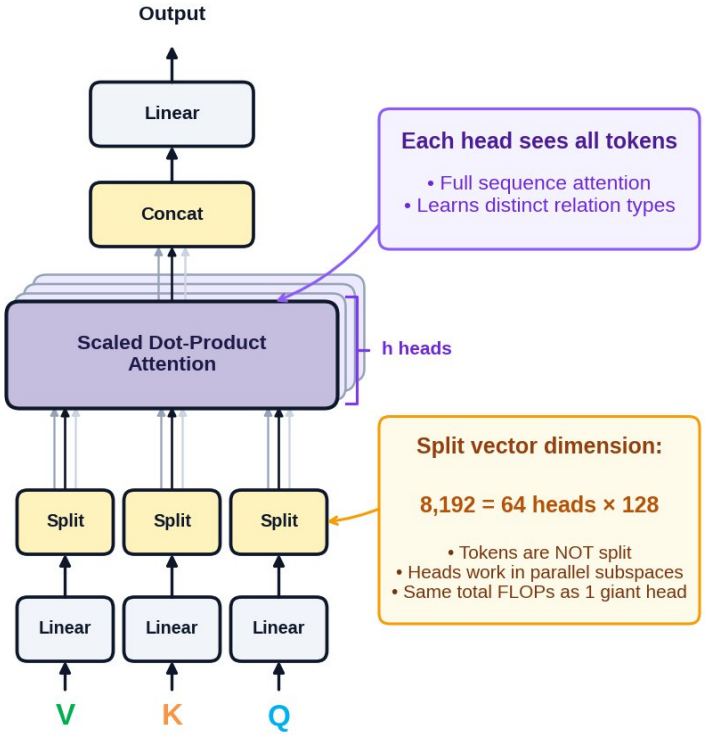
softmax : scaling & normalization
Each row → weights ≥0, sum=1 (attention distribution)

× V : weighted combination of values
Each row → weights ≥0, sum=1 (attention distribution)

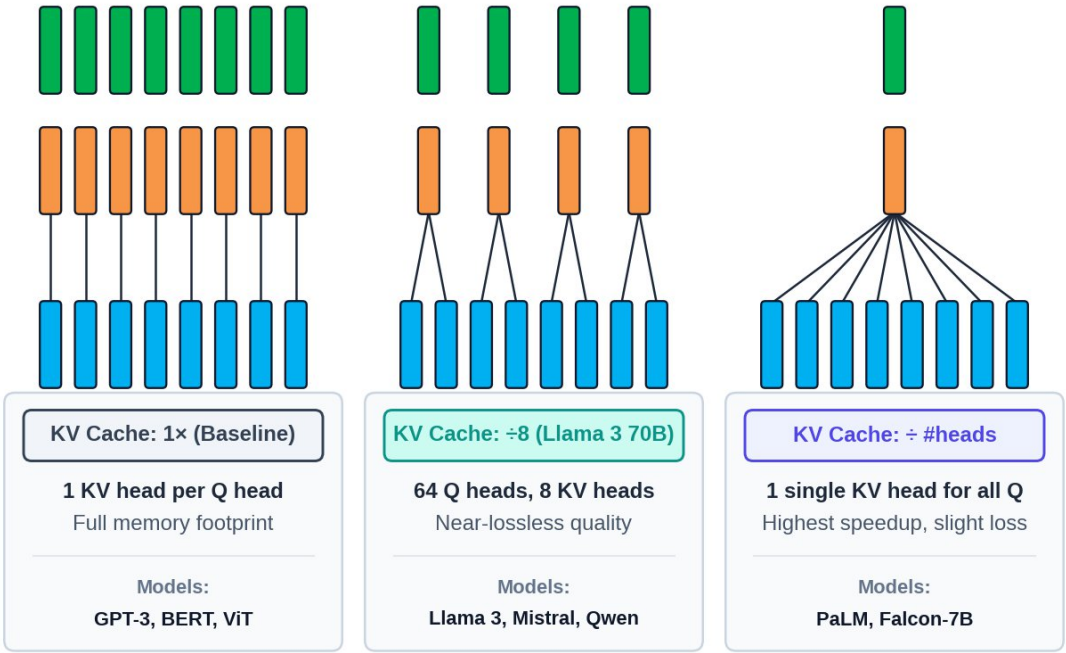
Quadratic complexity:
n × n attention matrix → $O(n^2)$ cost

Multi-Head Attention… and GQA

Architecture

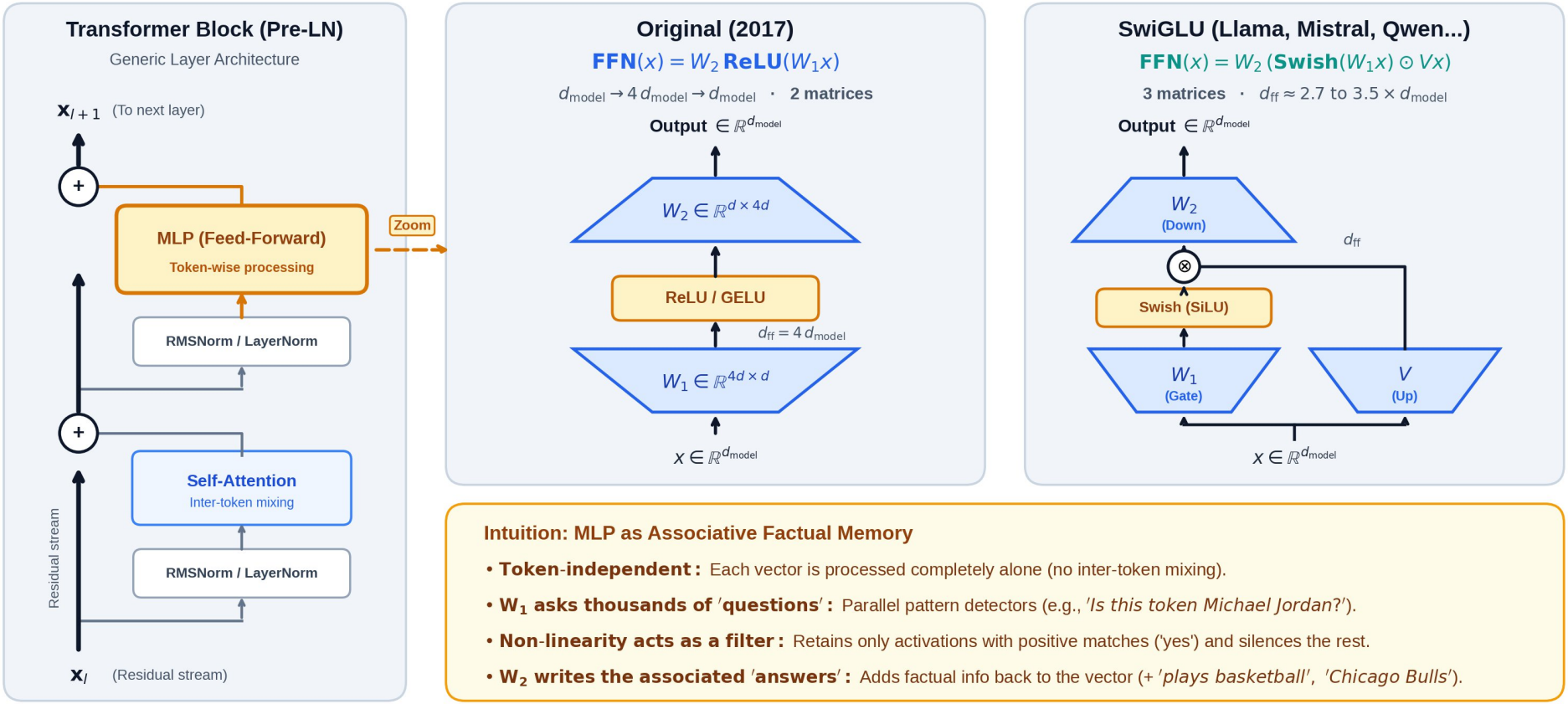


Attention variants



Fewer K, V heads → Smaller KV cache in generation (cf. KV cache bottleneck)

The MLP block (Feed Forward)



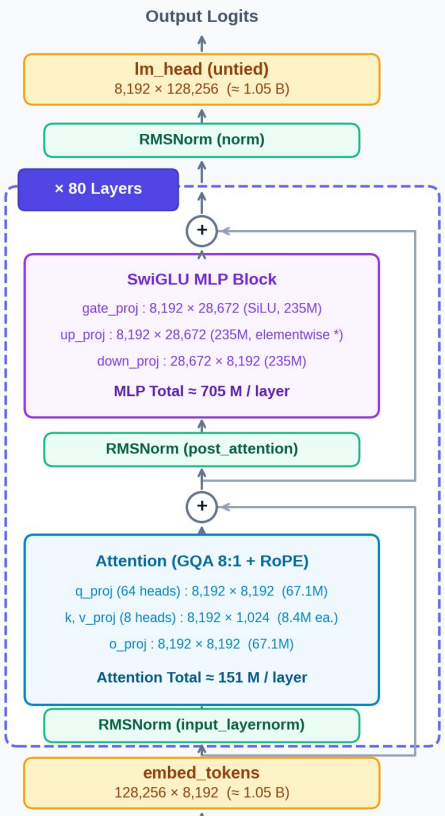
Intuition: MLP as Associative Factual Memory

- **Token-independent :** Each vector is processed completely alone (no inter-token mixing).
- **W_1 asks thousands of 'questions' :** Parallel pattern detectors (e.g., *'Is this token Michael Jordan?'*).
- **Non-linearity acts as a filter :** Retains only activations with positive matches ('yes') and silences the rest.
- **W_2 writes the associated 'answers' :** Adds factual info back to the vector (+ *'plays basketball'*, *'Chicago Bulls'*).

Parameter Budget: MLP accounts for ~2/3 of parameters in GPT-3, ~80% in Llama 3.3-70B

Where are the parameters? — Llama 3.3-70B

Llama Architecture Block



Hugging Face config.json — 7 essential hyperparameters

```
{
  "hidden_size":           8192,    Hidden state dimension (d)
  "intermediate_size":     28672,   SwiGLU intermediate dim (3.5 × d)
  "num_hidden_layers":     80,      Total transformer layers
  "vocab_size":            128256,  Vocabulary size (untied)
  "num_attention_heads":   64,      Query attention heads
  "num_key_value_heads":   8,       Key/Value heads (GQA 8:1 sharing)
  "head_dim":              128      Head dimension (8192 / 64)
}
```

Parameter Accounting (Hugging Face Names)

1. **embed_tokens :** vocab_size × hidden_size = 128,256 × 8,192 — **≈ 1.05 B**

2. **Attention (per layer) :** **≈ 151.0 M / layer**
   - q_proj, o_proj : 2 × (8,192 × 8,192) = 2 × 67.1 M = 134.2 M
   - k_proj, v_proj : 2 × (8,192 × 1,024) = 2 × 8.4 M = 16.8 M

3. **SwiGLU MLP (per layer) :** **≈ 704.6 M / layer**
   - gate_proj, up_proj : 2 × (8,192 × 28,672) = 2 × 234.9 M = 469.8 M
   - down_proj : 1 × (28,672 × 8,192) = 1 × 234.9 M = 234.9 M

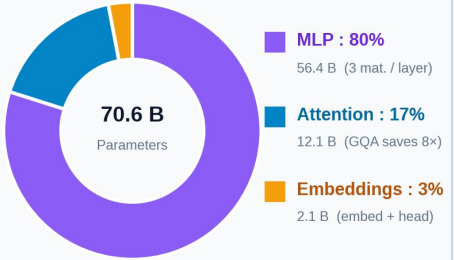
4. **Across 80 Layers (× 80) :**
   - Attention : 80 × 151.0 M **≈ 12.1 B**
   - MLP : 80 × 704.6 M **≈ 56.4 B**

5. **lm_head (untied) :** hidden_size × vocab_size = 8,192 × 128,256 — **≈ 1.05 B**

Total Parameters : ≈ 70.6 B ✓

1.05 B (embed) + 12.1 B (attention) + 56.4 B (MLP) + 1.05 B (head)

Parameter Breakdown



Key Takeaway: 80% of parameters reside in the MLPs!

HPC Memory Footprint — Jean Zay / IDRIS

Inference (BF16 Weights)

- 70.6 B × 2 bytes ≈ 141.2 GB
- → **Needs at least 2 × 80 GB GPUs**

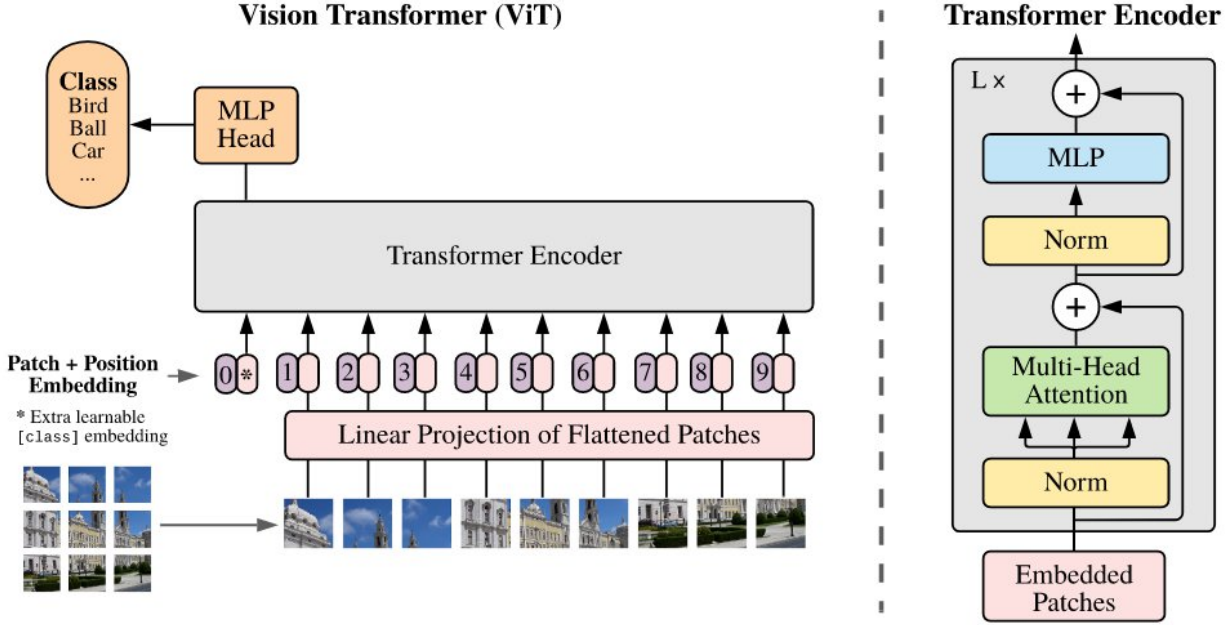
(weights only, before KV cache & activations)

Training (Adam + Mixed Precision)

- ~16 bytes / param (weights, grads, optimizer)
- 70.6 B × 16 bytes ≈ 1.13 TB
- → **Needs at least 15-16 × 80 GB GPUs**

(excluding activations — requires TP / PP parallelism)

Vision Transformer (ViT, 2020)



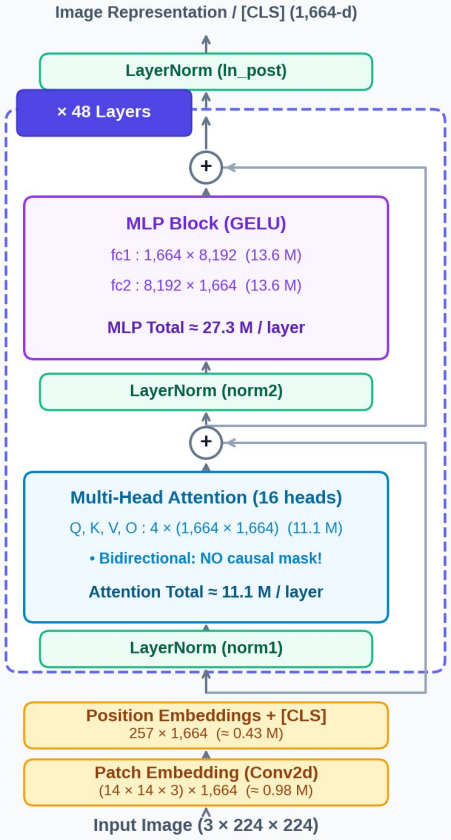
- Patches are the ‘words’ of the image (« An Image is Worth 16x16 Words »)
- + learned position
- Token [CLS] (like BERT): its output is used for classification
- Encoder only: no mask; each patch sees the entire image

Resolution ×2 → tokens ×4 → attention ×16

[12]

Where are the parameters? — ViT-G/14 (1.84 B)

ViT Encoder Block



OpenCLIP config.json

— ViT-bigG/14 (same dimensions as ViT-G/14)

```
{
  "hidden_size":           1664,   Hidden state dimension (d)
  "intermediate_size":     8192,   MLP dimension (4.92 × d)
  "num_hidden_layers":     48,     Total Transformer encoder layers
  "patch_size":            14,     Patch resolution (14 × 14 px)
  "num_attention_heads":   16,     Q/K/V heads (head_dim = 104)
  "num_channels":          3,      Input image channels (RGB)
  "image_size":            224     Input image resolution (16×16)
}
```

config: CLIP ViT-bigG/14 (OpenCLIP), identical dimensions to ViT-G/14 (Zhai et al., 2021)

Parameter Accounting (Vision Transformer)

1 — Input Embeddings (Patch + Positions) : ≈ 1.41 M (< 0.1%)

- Patch embed : (14 × 14 × 3) × 1,664 = 588 × 1,664 ≈ 0.98 M (3 RGB)
- Position embed : 257 × 1,664 = (16×16 patches + [CLS]) ≈ 0.43 M

2 — Attention (per layer) — Full MHA (no GQA) : ≈ 11.08 M / layer

- Q, K, V projections : 3 × (1,664 × 1,664) = 3 × 2.77 M = 8.31 M
- Output projection : 1 × (1,664 × 1,664) = 1 × 2.77 M = 2.77 M

3 — Classical MLP (per layer) — 2 matrices + GELU : ≈ 27.26 M / layer

- fc1 (expand) : 1 × (1,664 × 8,192) = 1 × 13.63 M = 13.63 M
- fc2 (project) : 1 × (8,192 × 1,664) = 1 × 13.63 M = 13.63 M

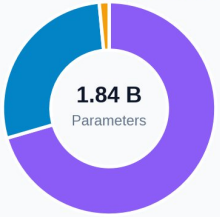
4 — Across 48 Layers (× 48) : ≈ 1.84 B / model

- Attention (48 layers) : 48 × 11.08 M ≈ 0.53 B (28.9% of model)
- MLP blocks (48 layers) : 48 × 27.26 M ≈ 1.31 B (71.1% of model)

Total Parameters : ≈ 1.84 B ✓

1.41 M (embeddings) + 0.53 B (attention) + 1.31 B (MLP) = 1,841.7 M

Parameter Breakdown



- **MLP : 71%** — 1.31 B (2 mat. / layer)
- **Attention : 29%** — 0.53 B (full MHA, 4 mat.)
- **Input : < 0.1%** — 1.4 M (patch + pos)

vs. Llama 70B (Vision vs. Language)

- **No vocabulary table :** In & out nearly free (< 0.1% vs 2.1 B in Llama)
- **38× fewer parameters (1.84 B vs. 70.6 B)**
- **Token explosion with image resolution :** At 518²: 37 × 37 = 1,369 tokens / image → heavy O(N²)

HPC Memory Footprint

Jean Zay / IDRIS

Inference (BF16 weights) :

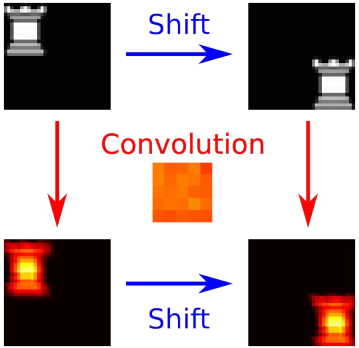
- 1.84 B × 2 bytes ≈ 3.7 GB → easily fits on 1 GPU!

Training (Adam + Mixed Precision) :

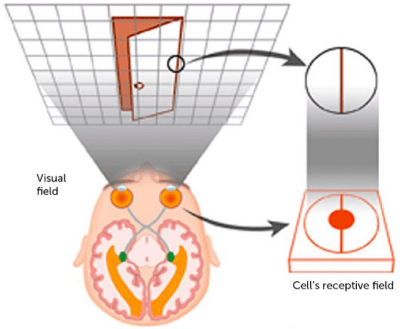
- Optimizer states: ~30 GB (~16 B / param)
- **High token count (1,369 at 518² resolution)**
  → Activations quickly saturate memory!
- ViT-G training: > 30,000 TPUv3 core-days (Zhai et al., 2021)

CoAtNet (2021): Convolution + Attention

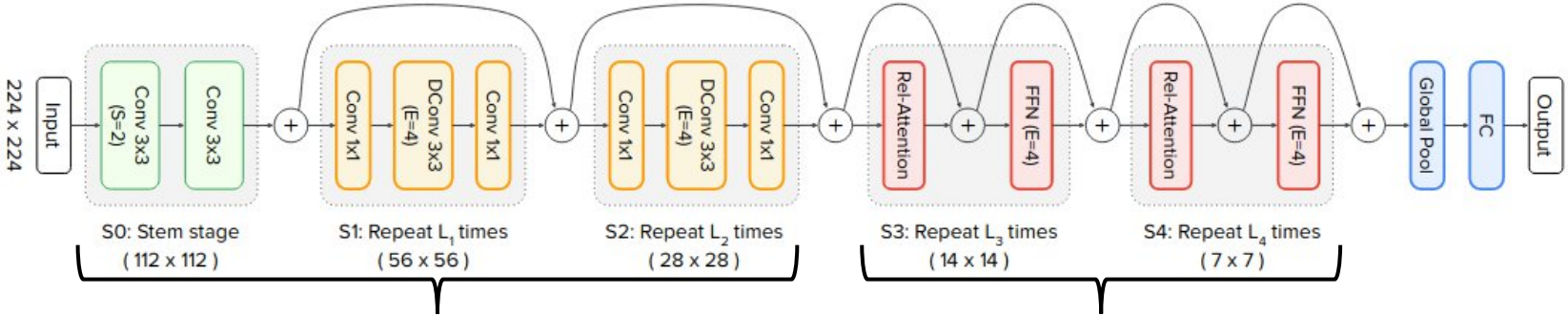
Convolution: looks locally, same filter everywhere → learns fast, with little data



Attention: looks at whole image → more expressive, but data-hungry



Input = raw image, no patches. Each "pixel" of the 14×14 map becomes a token



Convolutions (MBConv): high resolution, local features

Transformer : reduced map (14×14, then 7×7) → few tokens, global context

Replaces a CNN (ResNet) in a code without changing anything

CoAtNet: Sizes and Results (2021)

	Models	Eval Size	#Params	#FLOPs	TPUv3-core-days	Top-1 Accuracy
JFT-300M pre-training	ResNet + ViT-L/16	384^2	330M	-	-	87.12
	ViT-L/16	512^2	307M	364B	0.68K	87.76
	ViT-H/14	518^2	632M	1021B	2.5K	88.55
	NFNet-F4+	512^2	527M	367B	1.86K	89.2
	CoAtNet-3†	384^2	168M	114B	0.58K	88.52
	CoAtNet-3†	512^2	168M	214B	0.58K	88.81
	CoAtNet-4	512^2	275M	361B	0.95K	89.11
	CoAtNet-5	512^2	688M	812B	1.82K	89.77
JFT-3B	ViT-G/14	518^2	1.84B	5160B	>30K◇	90.45
	CoAtNet-6	512^2	1.47B	1521B	6.6K	90.45
	CoAtNet-7	512^2	2.44B	2586B	20.1K	**90.88**

JFT = Google's internal dataset (300 M then 3 B images)
evaluation on ImageNet-1k (1.28 M images)
ImageNet-21k: ~14 M images

same accuracy (90.45%), 3.4× fewer FLOPs,
> 4.5× fewer training computations

Key Takeaways: Transformers



- Entire sequence in parallel (≠ RNN), and each token sees all the others (≠ CNN, local)
- Attention = communication between tokens (cost in n^2); MLP = processing per token (majority of the weights)
- Scale very well: from 65 million (2017) to hundreds of billions of parameters
- Few prior assumptions → require very large datasets
- **No longer fit on a single GPU: parallelism (training), optimizations (inference)**

Optimizing Inference for Very Large Models

Less memory, more speed… without (too much) loss in quality

Distillation ◂

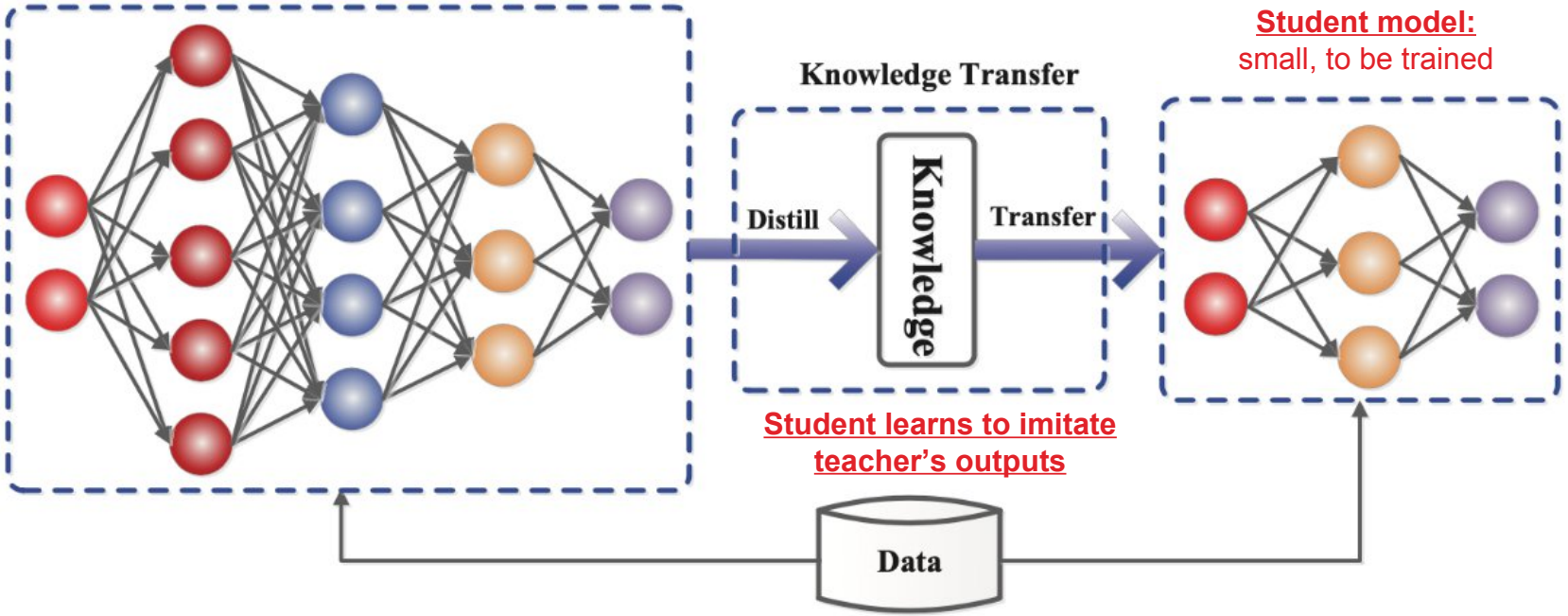
Quantization ◂

Pruning ◂

KV cache & PagedAttention ◂

Distillation: from big teacher to small student

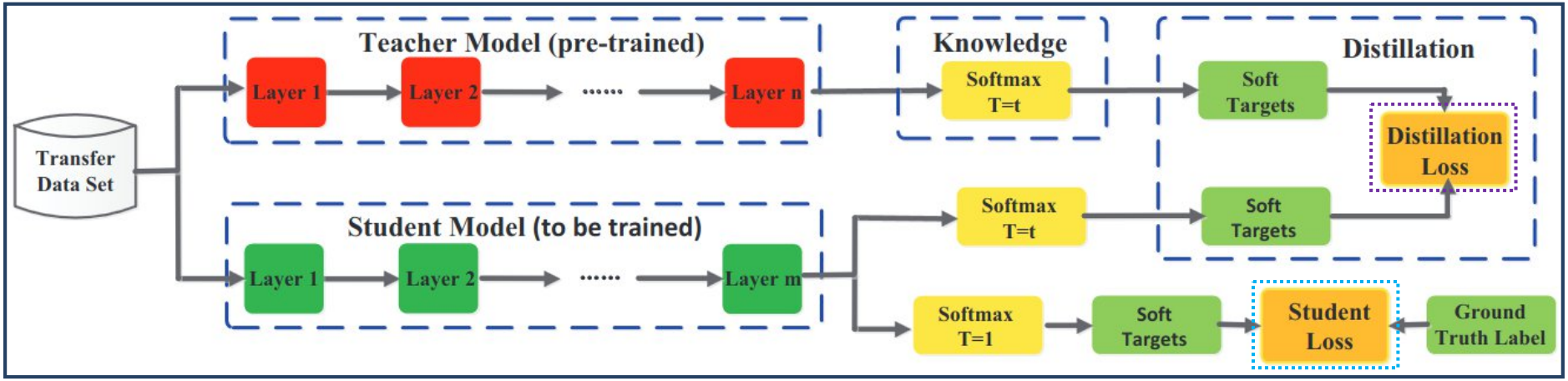
Teacher model:
big, already trained, fixed



Student model:
small, to be trained

Examples:
DistilBERT (2019) • DINOv3 (2025) : ViT-7B → ViT-B/L, ConvNeXt • Llama 3.2 1B/3B (2024) • DeepSeek-R1-Distill (2025)

Distillation: loss function



$$\mathcal{L} = \mathcal{L}_{\text{distil}}\left(\text{softmax}\left(z_{\text{teacher}}/T\right),\ \text{softmax}\left(z_{\text{student}}/T\right)\right) + \lambda\,\mathcal{L}_{\text{CE}}\left(y_{\text{true}},\ y_{\text{student}}\right)$$

KL (most common), cross-entropy, MSE sur les logits

Temperature Scaling in Softmax : softmax(z / T)

Class	Logit	T = 1 (Standard Softmax)	T = 2 (Distillation Softmax)	
Cat	4	87 %	67 %	
Dog	2	12 %	24 %	2× higher!
Car	0	1.6 %	9 %	

Ex.: DistilBERT,
40 % less params, 60 % faster,
97 % of the performances of BERT

Quantization: why?

Format	Bytes	A100	H100 SXM
FP32	4	19.5 TFLOPS	67 TFLOPS
TF32 (Tensor Core)	4	156	495
BF16 / FP16	2	312	989
FP8	1	—	1,979
INT8	1	624 TOPS	1,979 TOPS
Memory bandwidth		2.0 TB/s	3.35 TB/s

- Bits ÷2 → memory ÷2 and Tensor Core throughput ×2
- With 2:4 pruning, another ×2 (see pruning later); Blackwell: FP4

HPC for LLMs:
In the generation stage, all weights are re-read for each token
→ bandwidth is the limiting factor.
70 B in BF16 = 141 GB read per token; in 4-bit ≈ 35 GB
→ up to ~4× faster

Quantizing weights: how?



$$s = \frac{\max|w|}{127}, \qquad q = \text{round}\left(\frac{w}{s}\right), \qquad \hat{w} = s \cdot q$$

Example : $s = 0.01$; $w = 0.4237 \rightarrow q = 42 \rightarrow \hat{w} = 0.42$

⚠ Outliers Issue

- **A single weight at** $12.7 \rightarrow s = 0.1$
  → all small weights collapse to 0 **or** ± 1 **!**
- **Solution: One scale per row or per group**
  (e.g., separate scale for every 128 weights)

- **PTQ (Post-Training Quantization):**
  From a few minutes to a few hours,
  with a little calibration data
  (GPTQ, AWQ, GGUF, FP8...)

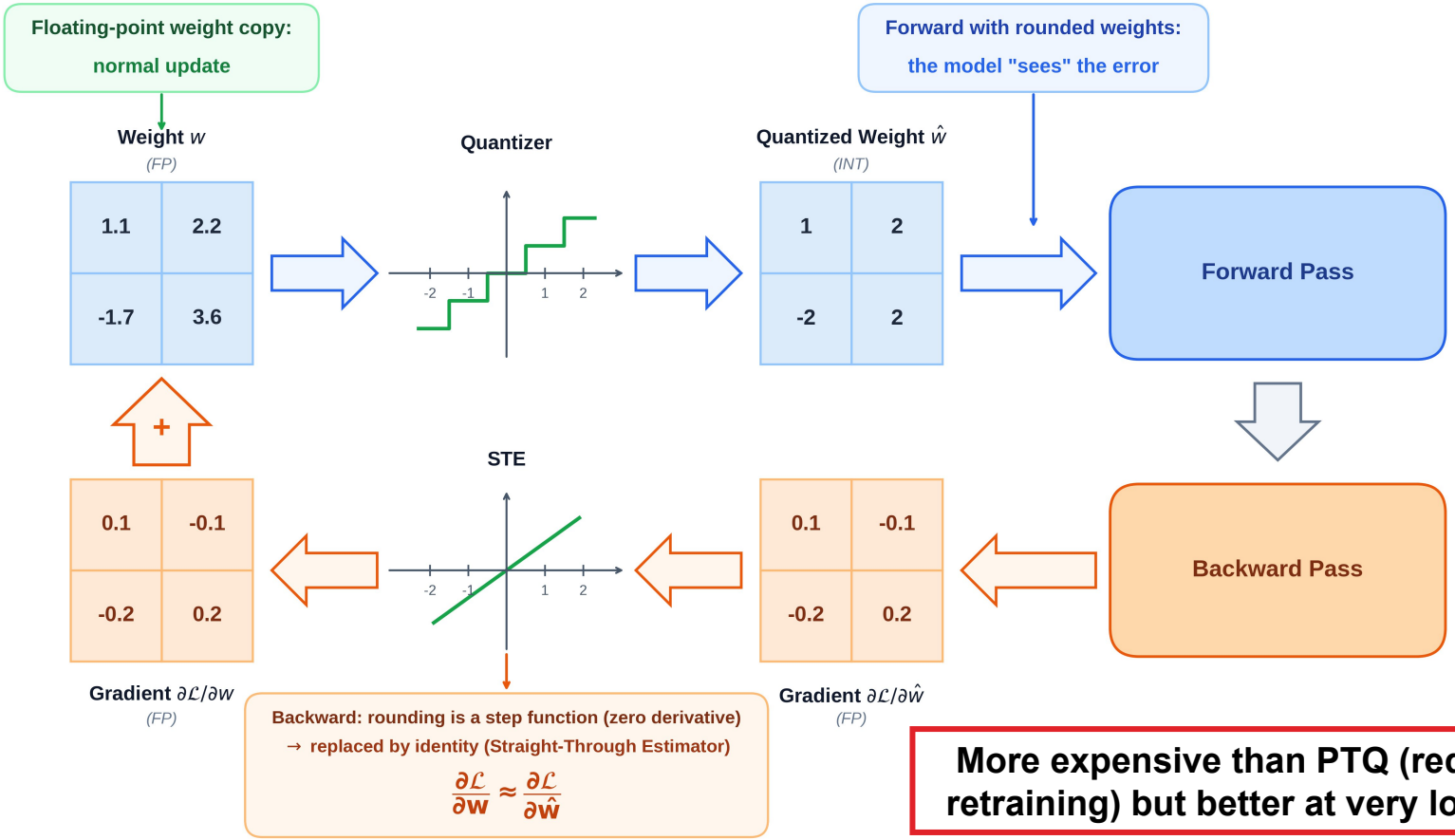
- **Precision Trade-offs:**
  8 bits ≈ lossless
  4 bits: small loss, very widely used
  < 4 bits: degrades rapidly

- **Quantization Scope:**
  Weights only (W4A16) → memory savings
  Weights + activations (W8A8, FP8) → compute speedup too

Always re-evaluate the quantized model!
Performance drops can occur depending on tasks.

Quantization during training

Quantization aware training (QAT)



More expensive than PTQ (requires retraining) but better at very low bits

Pruning: the lottery ticket

The Lottery Ticket Hypothesis.
A randomly-initialized, dense neural network contains a subnetwork that is initialized such that—when trained in isolation—it can match the test accuracy of the original network after training for at most the same number of iterations.

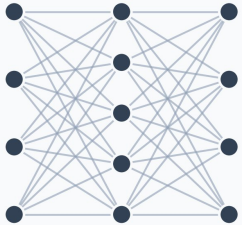
Frankle & Carbin, 2019

[24, 25, 26]

Pruning: the lottery ticket

STEP 1

Random Init (θ_0)



Dense Network

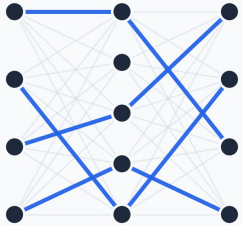
"Buys" millions of tickets

Random Init θ_0

Train

STEP 2

Train Network



Convergence to θ^*

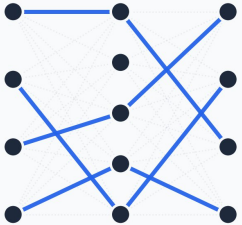
Most weights do very little!

100% Baseline Accuracy

Prune

STEP 3

Prune Weights



Cut Lowest 80–90%

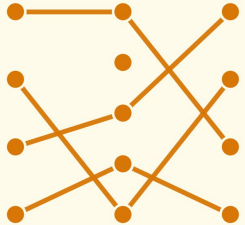
Isolate sparse mask m

10–20% Weights Left

Rewind

STEP 4

Rewind to θ_0



Reset to original θ_0

Ready to train in isolation

WINNING TICKET

THE PROOF: RETRAINING THE SPARSE MASK

Training the isolated subnetwork (10–20% weights) from scratch:

Rewound to Original Init (θ_0)

Matches 100% test accuracy • Equal training speed

New Random Init (θ'_0)

Severe accuracy drop • Proves lucky weights matter, not just sparsity

KEY TAKEAWAYS & IMPLICATIONS

What this means for deep learning in practice:

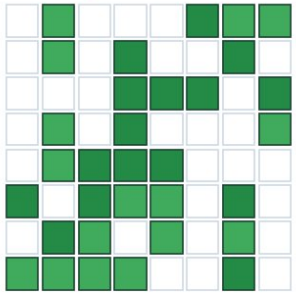
- **Why overparameterize?**
  Bigger models buy more lottery tickets (higher chance of a winning init).
- **Large Sparse > Small Dense:**
  At equal parameter count, a large pruned net beats a small dense net.
- **The Practical Limitation:**
  Must train dense net first: speedup applies to inference, not training.

Pruning: why?

1. Unstructured

Smallest weights, anywhere

Fine-grained · 50% zeros



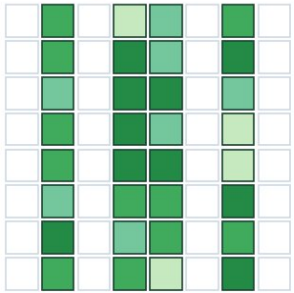
+ More accurate (minimal quality drop)

− Little or no GPU speedup

2. Structured

Entire rows or columns (neurons, heads, layers)

Coarse-grained · 50% zeros



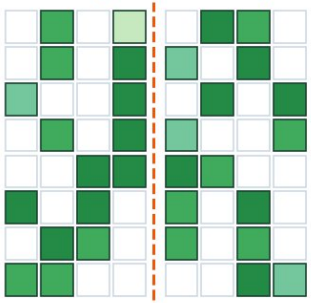
+ Truly smaller dense model (standard GEMM)

− Higher accuracy loss

3. Semi-Structured (2:4)

2 zeros per group of 4 contiguous weights

2 of 4 kept 2 of 4 kept



+ GPU-accelerated (NVIDIA Tensor Cores)

→ Best compromise (→ next slide)

Pruning criterion: which weights to drop?

- **Simplest criterion, often as good: weight magnitude $|w|$**

  Pruning smallest $|w|$ is fast and competitive with complex methods [25].

- **For LLMs: magnitude × activation size: $|w| \times |x|$ (cf. Wanda)**

  ~50% zeros in one pass without retraining (e.g. Llama 3.2 1B/3B from 8B).

Key questions

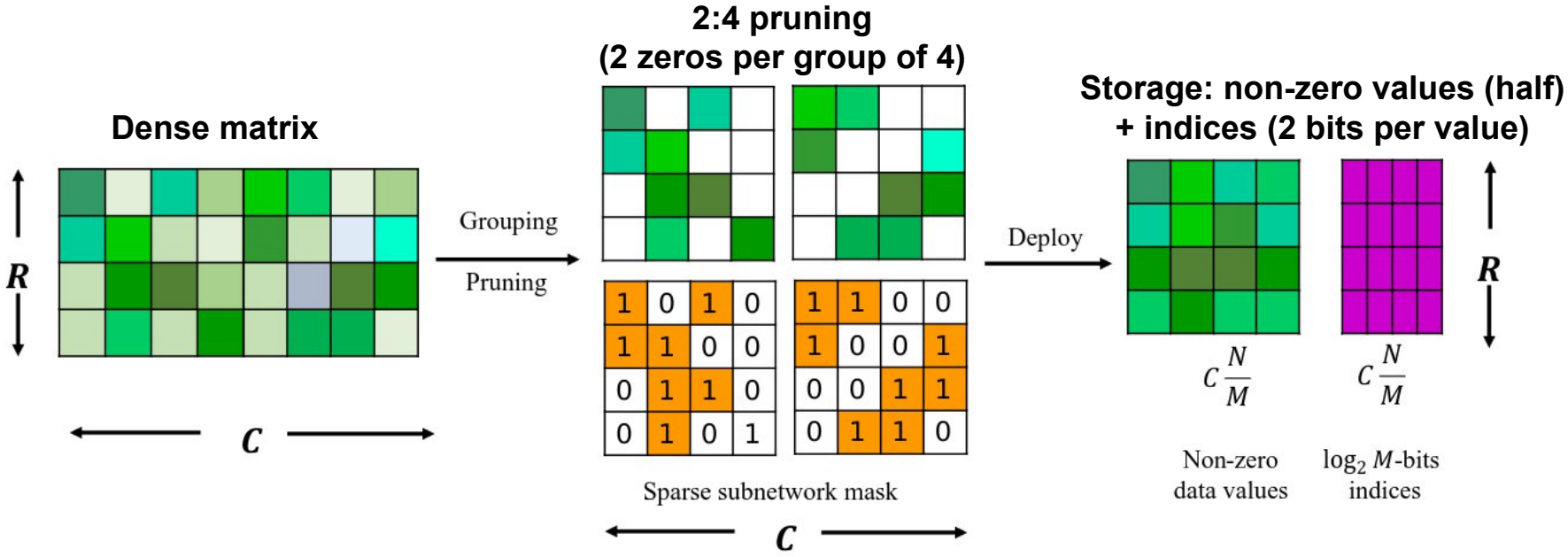
- **How much to prune?**

  How many weights can be zeroed? (fine-tuning + STE [20])

- **What real impact on compute time?**

  Theoretical FLOP reduction ≠ actual GPU speedup!

2:4 Pruning on NVIDIA GPUs

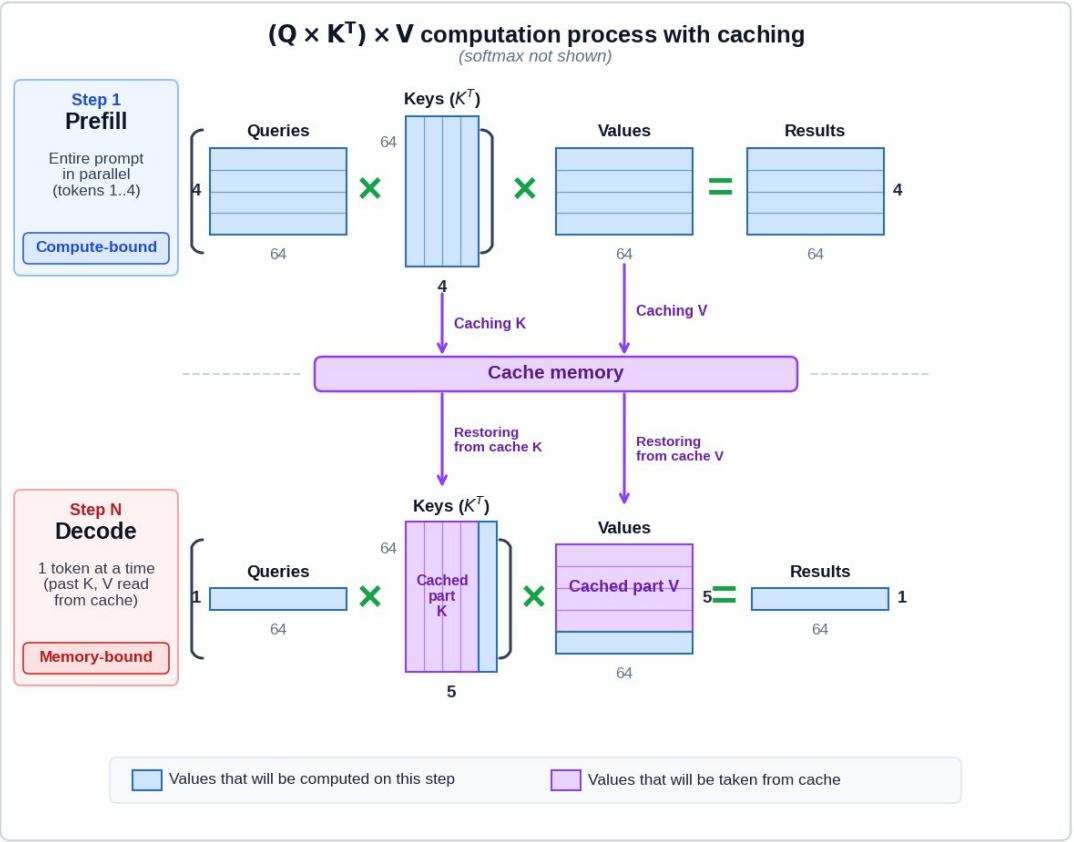


- A100/H100 Tensor Cores skip zeros → up to ×2 on matmuls
- Weights ~÷2 in memory ; combinable with INT8 / FP8
- Actual end-to-end gain < ×2, and a loss of precision that needs to be recovered (fine-tuning)”

KV Cache

"The big red" → Model → "dog" → "The big red dog" → Model → "sleeps" ...

Do we need to recompute everything at each token?



CAUSAL MASK

Past K and V never change

→ **We keep them in cache!**

HPC: KV CACHE MEMORY FOOTPRINT

$$\text{KV cache} = 2 \times n_{\text{layers}} \times n_{\text{KV_heads}} \times d_{\text{head}} \times \text{bytes} \times n_{\text{tokens}}$$

$2 \times (K, V)$ • 2 bytes (bf16) • n_{tokens} = sequence length

Llama 3.3-70B in bf16 :

(80 layers, 8 KV heads with GQA, head dim 128)

$2 \times 80 \times 8 \times 128 \times 2$ B ≈ **0.33MB/token**

→ **128k tokens ≈ 43 GB per sequence!**

(without GQA : ×8 ≈ 344 GB)

→ **Exceeds half an 80 GB GPU for just 1 sequence!**

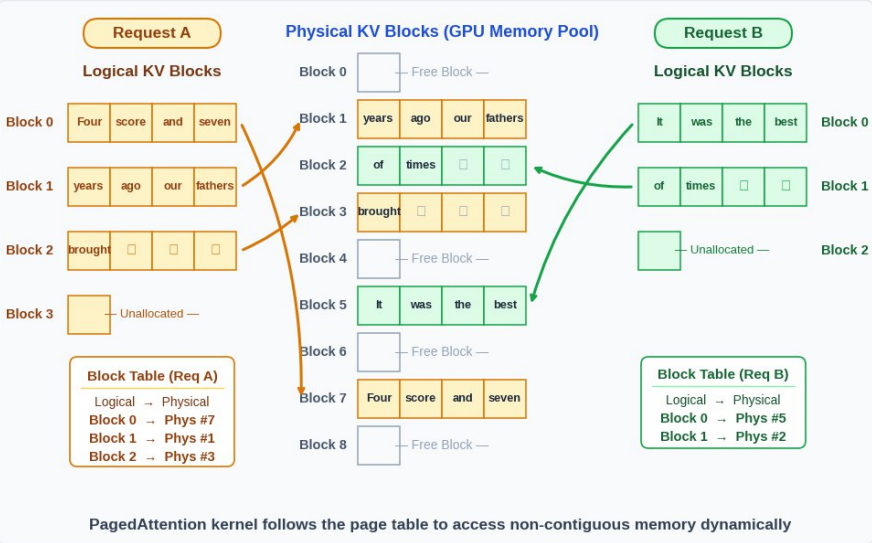
Paged Attention

Before: reserving contiguous max sequence length upfront → 60–80% KV cache wasted (reserved, fragmented)



Legacy systems pre-allocated memory for the maximum sequence length upfront → GPU VRAM filled with empty reserved placeholders

vLLM: small blocks (e.g. 16 tokens) placed anywhere + logical → physical table (like OS pages) → < 4% waste



Shared blocks:
An identical prompt (system prompt, beam search, multiple responses) is stored only once in GPU memory.

More parallel requests:
Near-zero waste packs more requests in VRAM → 2×–4× throughput boost (vs. 2023 serving systems).

vLLM — Multi-GPU Model Parallelism (Jean Zay)

```
from vllm import LLM, SamplingParams
llm = LLM(model="meta-llama/Llama-3.3-70B-Instruct",
          tensor_parallel_size=4)        # 1 node 4×H100
outputs = llm.generate(prompts,
                       SamplingParams(temperature=0.7, max_tokens=256
```


Kwon et al., SOSP 2023 — now ubiquitous (vLLM, SGLang, TensorRT-LLM, TGI)

Recap : which technique for which problem

Technique	What it reduces	Typical gain	Cost
Distillation	number of parameters	model several times smaller	train the student (+ have the teacher run it)
Quantization	bits per weight (16 → 8 → 4)	memory ÷2 to ÷4, faster generation	loss of precision (at 4 bits and below)
Pruning	non-zero weights	2:4 → up to ×2 on matmuls	limited real-world gain, fine-tuning
GQA	KV cache size	÷8 (Llama 3 70B)	architecture choice, before training
KV cache	calculations redone for each token	much faster generation	memory (0.33 MB/token for Llama 70B)
PagedAttention	wasted KV cache	throughput ×2 - 4	none (just use vLLM)

Cumulative: e.g., distilled model + 4 bits + vLLM